

Lecture X - Tooling, Git and Your Project

Programming with Python

Dr. Tobias Vlček

Kühne Logistics University Hamburg - Fall 2026

Checkpoint 5

Sessions VIII-IX. The first 40 minutes are the checkpoint. It starts now: the board runs one final audit before the big news.

- **Individual work:** no neighbors, no chat
- **AI tools are allowed:** being able to **VERIFY** the output is the skill being graded
- The **link is on Moodle**. Open it and start
- 6 short tasks: the right number out of a pandas table, a line the AI got wrong, a correct read of a chart
- It sweeps **Sessions VIII-IX:** pandas filtering and `groupby`, fair charts vs. misleading ones

When you're done

Menu → *Download* → *Download Python code* → upload the `.py` to the “**Checkpoint 5**” assignment on Moodle. **No retakes:** one sitting.

...

Note

The green live checks are **provisional**. The final grading runs on my side. The two multiple-choice tasks only say “*recorded*”, never correct or wrong. That’s expected, they’re scored later. Everything in it was rehearsed in the labs.

Episode 10: The Exit

The deal closes

Pens down. The checkpoint is behind you, and so, it turns out, is the startup. Overnight the rival chain **MunchCorp** stopped competing and made an offer: they’re **buying the company**. The investor read the term sheet, did the math against her own, and told you to take it: her money would have bought growth, theirs buys the whole shop.

...

The lawyers will tell you deals close on the numbers. This one closed on **yours**. The clean order data you pulled together in the data room (Session VIII) and the dashboard

you built for the pitch (Session IX) are exactly what convinced the acquirer the books were real.

...

No cliff-edge growth chart with a chopped axis. Just data that survived a second look. **That's why they signed.**

The handover note

Tobi's job today is to hand over the **data room**: every file the company ever built. He tapes a note to the folder:

...

"It's all here. And it was never anything fancy. Every notebook you downloaded all semester was already a plain `.py` file. Same code, same lines. Open one in a real editor and it just runs."

...

That's the whole reveal, and most of you already suspected it. Those downloads stop being **backups** today and become the **real thing**: on your own machine, in a real editor, under your control.

Your exit package

Every acquisition comes with an exit package. Here's yours: you don't join MunchCorp. You **spin off your own idea** and build it.

...

- The tools are the same ones professionals use: uv, Zed, git, GitHub
- The work is a real project, built with a partner, over the next weeks
- Today we set up the machine and the repo; the building starts here

...

Let's get you kitted out.

The Project

The brief

Over the next weeks you build **one real project** and present it in the final session.

- Work in **pairs**: solo is fine if the numbers don't come out even
- **Deliverable**: a **GitHub repository link**, submitted on Moodle
- The repo's `README.md` carries a short **AI-disclosure section**: which tools you used, what for, and what you **verified yourself**
- You **present in Session XIII**: 10 minutes, plus 5 minutes for questions

...

💡 Tip

Pick something you're **actually interested in**. You'll spend real hours on it. Curiosity is the fuel that carries a project past the boring middle.

How it counts

- Project **and** presentation together are worth **40 of the 100** course points
- Your **commit history counts**: it's the evidence of *how* the work happened, which the final state alone can't show. Commit small and often (git block, shortly)
- **Sessions XI and XII are supervised work sessions**: I'm in the room while you build, so bring your questions

...

i Note

The project does not have to be flawless. Show that you scoped it realistically, built it in the open, and can explain your own code. That's the whole bar.

Idea 1 & 2

Data analytics dashboard

- **Core**: load and clean a real CSV with pandas, answer **3 business questions** with charts

Bonus: interactive dashboard (streamlit), automated reporting.

...

Web scraping pipeline

- **Core**: scrape **one** site politely into a CSV and analyze it

Bonus: multi-source, change detection, alerts.

Idea 3 & 4

Simulation study

- **Core**: a Monte Carlo simulation of a business decision (pricing, staffing, inventory) with matplotlib results

Bonus: interactive parameters, scenario comparison.

...

Automation assistant

- **Core**: a script that does a real repetitive task **end to end**, with error handling and logging

Bonus: scheduling, notifications, a small UI.

Idea 5 & 6

Game

- **Core:** a complete playable **terminal or pygame** game with save/load

Bonus: levels, high scores, procedural content.

...

ML-powered app (*ambitious: lean on AI*)

- **Core:** train a **simple** model on tabular data and wrap it in a small interface

Bonus: monitoring, A/B comparison.

Idea 7 & 8

Your startup spin-off

- **Core:** take any system from the course universe (courier routing, review moderation, demand forecasting) and build the **real** version

Bonus: wherever your version wants to grow.

...

Your own idea: the open door

- Have something else in mind? **Make it ambitious.** Bring it and we'll discuss scope and feasibility together.

Form your pair, pick your direction

Take ~25 minutes

Right now, in the room:

- **Find your partner** (or decide to go solo if the count is odd). No partner yet? **Come to the front**, I'll match you
- **Pick a direction** from the menu, or bring your own
- Talk it through: what's the *core* you can definitely finish, what's a *bonus* if time allows?

...

Tip

A good outcome by the end of this block: a one-sentence pitch ("we're building X that does Y for Z"), a name for the project, and a decision on who owns the shared repo (the owner creates it in the git block; the partner clones). The sentence keeps you honest about scope; the name goes on the repo in a minute.

Your Toolchain

Check your install

Before we build anything, the installs from the Session IX homework need to work. In a terminal:

```
git --version
gh --version
uv --version
```

- Three version numbers? You're set.
- Zed opens, and **Mistral Vibe** shows up in its agent panel? Good.

...

! Important

Didn't work? Don't burn the session fighting it. Flag it, pair up with your partner's working machine, and follow along on the slides for now. I sort it out with you in class or by e-mail; nobody gets left behind.

Once per machine

Before your laptop can talk to GitHub, it has to prove it's allowed to. Signing in to **Zed** **does not do this**; git needs its own login, and the `gh` tool handles it. Once per computer, in a terminal:

```
gh auth login
```

Choose GitHub.com, **HTTPS**, authenticate in the browser, and answer yes to “Authenticate Git with your GitHub credentials?”. Then, once:

```
git config --global pull.rebase false
```

...

! Important

Login fighting you? Same rule: **flag it**, follow along on the slides, and finish at home with the [Git Basics](#) page, or with me in the supervised sessions. Nobody's project stalls on a login.

Your project starts on GitHub

Everyone, right now, a **practice repo**. On github.com click **New repository**, then:

- **Name:** `python-practice`
- **Private**

- Tick **Add a README file**
- **.gitignore template:** `Python`

Create repository, then copy the **HTTPS URL** from the green Code button: `https://github.com/you/python-practice.git`

...

That's a **repository**: a folder git watches, with its whole history. So far it lives only on GitHub. Next we bring it down.

Bring it down: clone

In Zed's command palette (`Cmd/Ctrl+Shift+P`): **git: clone**, paste the URL, and pick the folder where you keep course work. Zed opens the new folder as your workspace: the file tree on the left shows `README.md` and `.gitignore`.

```
git clone https://github.com/you/python-practice.git
```

...

i Note

Everything from here on happens **inside this folder**: git, uv, the AI. If Zed ever shows an empty file tree, **File → Open Folder** and pick the cloned folder itself, not the one above it.

Make it a Python project: `uv init`

Open Zed's built-in terminal (`Ctrl+``); it starts inside your workspace. Then:

```
uv init --no-package
```

`uv init` adds `main.py`, `pyproject.toml` and `.python-version` next to the `README.md` and `.gitignore` that GitHub made.

...

i Note

`--no-package` keeps it a flat folder of scripts, the shape you know from the labs; without it, uv builds an installable package layout you don't need yet. And no `git init`: the folder came from GitHub, it **is** a git repository already.

Open a notebook in Zed

Grab a lab `.py` you downloaded in **Part I**: we'll use `nb_03_lab_functions.py`. Copy it into your project folder, then open it in Zed: it's just code. Read it.

The file needs its tools first; adding a package is one command:

```
uv add marimo
```

Now run it, no browser required:

```
uv run python nb_03_lab_functions.py
```

...

`uv run` uses the project's own Python and packages. The file that lived in a browser tab all semester now runs on **your machine, from your editor**. That's the whole point of today. (*Part-I labs only today; the pandas labs need more packages and their CSV.*)

Connect your AI

Part III **encourages** AI, and it is already installed: Mistral Vibe, running inside Zed on the key from Session VI (setup in the [AI-tools guide](#)). Open the agent panel, pick Mistral Vibe, and try it on your five-minute-old project:

Explain what `main.py` does, then add a docstring. Don't change anything else.

Read the diff Zed shows you **before** you accept it. That's the whole workflow: you ask, it drafts, you verify.

...

! Important

The disclosure rule still stands, now in writing: your repo's `README.md` says **which tools you used, what for, and what you checked yourself**. AI drafts; **you verify**, same reflex as the checkpoint you just sat.

Inside the panel: a model in a loop

- Mistral Vibe is an agent: the language model you met in Session VI, wired into a loop with tools
- The model **proposes** an action: read a file, edit a line, run a command
- The tool **runs** it, the result **feeds back**, and it proposes again, until the goal is met
- No magic, and no memory between sessions

...

Every failure mode from Sessions VI and VIII applies at every step, and a loop gives them many steps to compound in. The diff you just read is the guardrail.

Git

The five words

Git has a small vocabulary. Learn these five and you can do everything this project needs:

- **Repository**: a folder git watches, with its whole history
- **Commit**: one saved snapshot, with a short message
- **Push**: send your commits up to GitHub
- **Pull**: bring your partner's commits down to you
- **Clone**: make your own copy of a repo from GitHub

...

Two of them you've already done: the repo exists, and you cloned it. The other three now, **live, in your practice repo**. Mistakes are cheap when the repo is five minutes old.

Your first commit

Your folder has new files worth saving: `main.py`, `pyproject.toml`, `.python-version`, the lab. In Zed's **command palette**:

- **git: stage all**: mark everything as part of the snapshot
- **git: commit** (Cmd/Ctrl+Enter): type a message, save it

...

The same thing in the terminal, the words you'll see everywhere:

```
git add .
git commit -m "Set up the project"
```

A commit lives **only on your computer** so far. Next we send it to GitHub.

Push

Because the folder came from GitHub, git already knows where "up" is. In Zed: **git: push**. In the terminal:

```
git push
```

...

Reload the repo page on github.com: `main.py` is there. That's the loop you'll repeat all project long: **edit** → **commit** → **push**. Pull is the other direction; you'll see it in a minute.

The real project repo: owner only

Now the one that counts. **Repo owner only**, same recipe as the practice repo, with three extras:

1. Name it after your **pitch**; **Private**, README, Python `.gitignore`

2. **Settings → Collaborators:** invite your partner and me (my GitHub username is on Moodle), so I can read it for grading
3. Clone it in Zed, `uv init --no-package`, commit, push, exactly as before

...

Partner: **don't create a second project repo.** Yours arrives by invitation in a moment.

Your partner joins

The **partner** accepts the invitation (GitHub e-mails it) and clones the project **into a fresh folder**, next to the practice repo:

- In Zed: `git: clone`, paste the project's **HTTPS URL**

```
git clone https://github.com/owner/project.git
```

...

Cloning brings the whole project, its history and the connection to GitHub; the partner pushes and pulls normally from the first moment. Now the owner edits `README.md`, commits and pushes, and the partner runs `git: pull` (`git pull`): the change lands on the second laptop. That's the fifth word.

...

Note

Clock ran out? This step keeps: inviting and cloning works just as well at the start of **Session XI**, with me in the room.

How we work together

Two people, one repo. Four habits keep it painless:

- **Pull before you start.** `git: pull` first, every session: this alone prevents almost every problem
- **Commit small and often.** Little snapshots with clear messages beat one giant one
- **A conflict? Call me.** If git marks a clash, don't fix it alone in week one. Bring it to class
- **Never force anything.** No `--force`, no "force push", ever

...

Note

Everything today (the actions, the terminal equivalents, the setup, and a **cheatsheet table**) lives on the [Git Basics](#) page. When today's follow-along is a fading memory, that's the page you reopen.

Send-off

The road to the finish

- **Sessions XI and XII:** supervised work sessions. I'm in the room; you build, I help. Pull before you start, push when you stop.
- **Session XIII:** presentations. 10 minutes each, 5 minutes of questions. Show the idea, the build, and one thing that broke and how you fixed it.

...

Tip

Between now and then, the repo tells your story. Commit as you go. A steady history is worth more than a heroic final night.

AI and your first job

- Large experiments with thousands of developers: **+26%** tasks completed with AI, and juniors gained the most
- But experienced developers on their own codebases went **19% slower** while *feeling* 20% faster
- And a learning study: students coding with AI scored **50%** on a mastery quiz vs **67%** without, worst gap on **debugging**
- The same study family found the fix: guarded use, hints instead of finished solutions, erased the harm. Part I was exactly that

What stays valuable

- The verification boundary: usable delegation ends where your ability to **check** ends. Growing that boundary outward is your career
- Comprehension debt: code that works today but nobody understands tomorrow. The interest comes due when a reviewer asks "why does this work?"
- Your edge, deliberately kept: **explain every line before you accept it**, and code without AI sometimes

Discussion: your future with AI

Note

Three questions, five minutes, show of hands:

- Which task from the past weeks would you **never** delegate to an AI?
- What will an entry-level logistics analyst do in 2030, when AI drafts the first version of everything?
- If AI writes most junior code, where do the senior people come from?

Keep programming

The skill fades if it sits idle. A few ways to keep it alive:

- Use Python in your **thesis**: data cleaning, analysis, plots you can defend
- Find a way to **apply it at work**: the repetitive task nobody wants to do by hand
- **Advent of Code**: free, ad-free programming puzzles, one a day from **December 1st**.
A genuinely fun way to keep the muscle warm.

Thank you

- We covered the basics of Python: from a first `print` to a real project on your own machine
- You debugged, you verified AI, you shipped real numbers
- I hope you enjoyed it, and I hope it's useful long after the grade
- If you have questions or feedback, [please tell me](#)

...

All the best for your studies and your career. Now go build the thing.

Literature

Books to start with

- Downey, A. B. (2024). Think Python: How to think like a computer scientist (Third edition). O'Reilly. [Link to free online version](#)
- Elter, S. (2021). Schrödinger programmiert Python: Das etwas andere Fachbuch (1. Auflage). Rheinwerk Verlag.

...

Note

Setting up the toolchain? The [Installing Python](#), [AI Tools](#) and [Git Basics](#) pages are the reference versions of everything we did live today.

...

For more, see the [literature list](#) of this course.